

CareTrace: A Neurosymbolic Approach to Pediatric After-Hours Triage

Kirthi Shanbhag
UC Berkeley School of Information
kirthi_shanbhag@berkeley.edu

Niyanthri Naresh Prabhu
UC Berkeley School of Information
niyanthri.naresh@ischool.berkeley.edu

Sohail Khan
UC Berkeley School of Information
sohailk2@ischool.berkeley.edu

Xinran Zhang
UC Berkeley School of Information
zhangxr7@ischool.berkeley.edu

DATASCI 290 — Neurosymbolic AI | UC Berkeley School of Information | Spring 2026

ABSTRACT

After-hours pediatric triage is a high-stakes setting in which caregivers seek guidance for acutely ill children without immediate access to clinicians. Large language models (LLMs) are increasingly capable of producing plausible clinical language, but pure generative approaches lack verifiable reasoning/provenance, cannot guarantee adherence to established clinical practice guidelines (CPGs), and risk fabricating medication dosing or epidemic content. **CareTrace** is a neurosymbolic prototype that grounds pediatric triage in the Seattle Children's Fever CPG by combining (1) an LLM populated from the CPG's red-flag criteria, (3) deterministic PyDatalog triage rules with full audit trace, and (4) a provenance-annotated explanation agent. A LangGraph orchestration pipeline sequences these components into a multi-turn conversational system. On three manually crafted scenarios spanning home management, emergency escalation, LLM baseline cannot guarantee structured CPG adherence. The architecture demonstrates how neurosymbolic integration can improve safety, transparency, and auditability in conversational clinical decision support.

1. INTRODUCTION

1.1 Motivation

Pediatric fever is one of the most common reasons families contact medical advice after hours. Guidelines such as the Seattle Children's Fever CPG define clear escalation criteria — seizure, altered consciousness, breathing difficulty, extreme fever, infant under three months — that must be applied consistently. Parents describing symptoms in natural language often omit details critical for safe triage, and any system that fails to elicit those details risks either over-escalating (unnecessary ER visits) or under-escalating (missed emergencies).

Pure LLMs present two risks in this setting: (a) they cannot guarantee that CPG thresholds are honored because reasoning is implicit in weights rather than explicitly asserted, and (b) they can hallucinate — generating plausible-sounding but incorrect medication dosing, epidemiological context, or age-specific contraindications. Prior work in clinical NLP has shown that symbolic reasoning layers, when coupled with neural language understanding, yield more auditable and safer outputs in structured medical domains.

CareTrace was designed to demonstrate this neurosymbolic integration at prototype scale, with a deliberately bounded clinical scope: febrile illness, gastrointestinal symptoms, and dehydration risk in pediatric patients.

1.2 Contributions

- Neurosymbolic triage pipeline:** LangGraph state machine sequencing LLM interpretation, SNOMED-grounded KG retrieval, PyDatalog logic rules, and templated explanation with provenance strings.
- CPG-derived mini knowledge graph:** SNOMED CT concepts seeded from the Seattle Children's Fever CPG stored in Neo4j AuraDB with IS_A hierarchy and attribute edges.
- Audit-traceable reasoning:** every disposition includes a named `rule_ids` set explaining exactly which symbolic predicate fired.
- Medication safety hooks:** explicit CPG citations for acetaminophen and ibuprofen age gates with provenance strings tying every statement to the source document URL.
- Comparative baseline:** a vanilla LLM baseline without symbolic gates enabling direct evaluation of provenance and consistency differences.

2. BACKGROUND

2.1 Neurosymbolic AI

Neurosymbolic AI combines the pattern-recognition strengths of neural networks with the deductive reasoning guarantees of symbolic systems. In the clinical domain this combination is particularly valuable: neural components handle the variability and ambiguity of natural language, while symbolic components enforce hard constraints derived from domain knowledge. More recently, LLM-plus-symbolic frameworks have emerged for structured document question answering and clinical guideline compliance.

2.2 Clinical Practice Guidelines and Pediatric Triage

Clinical practice guidelines are evidence-based documents that define care protocols for defined conditions. The Seattle Children's Fever CPG specifies fever thresholds, escalation triggers, and medication guidance aligned with the American Academy of Pediatrics. Encoding CPG logic as machine-executable predicates is an established approach in clinical decision support systems (CDSS). CareTrace applies this approach in a conversational setting where intake fields must be elicited dynamically from unstructured caregiver language.

2.3 Knowledge Graphs for Medical Concept Grounding

SNOMED CT is the world's most comprehensive clinical terminology with over 350,000 active concepts hierarchically organized through IS_A and attribute relationships. Snowstorm is an open-source SNOMED CT terminology server that exposes FHIR-compliant REST APIs. Neo4j is a native property graph database widely used in biomedical knowledge graph research. Combining Snowstorm concept retrieval with Neo4j graph persistence enables compact, query-friendly SNOMED subgraphs that can ground natural language terms to standard clinical identifiers.

3. SYSTEM ARCHITECTURE

3.1 Overview

CareTrace is implemented as a Python package (`CareTrace`) with the following principal layers:

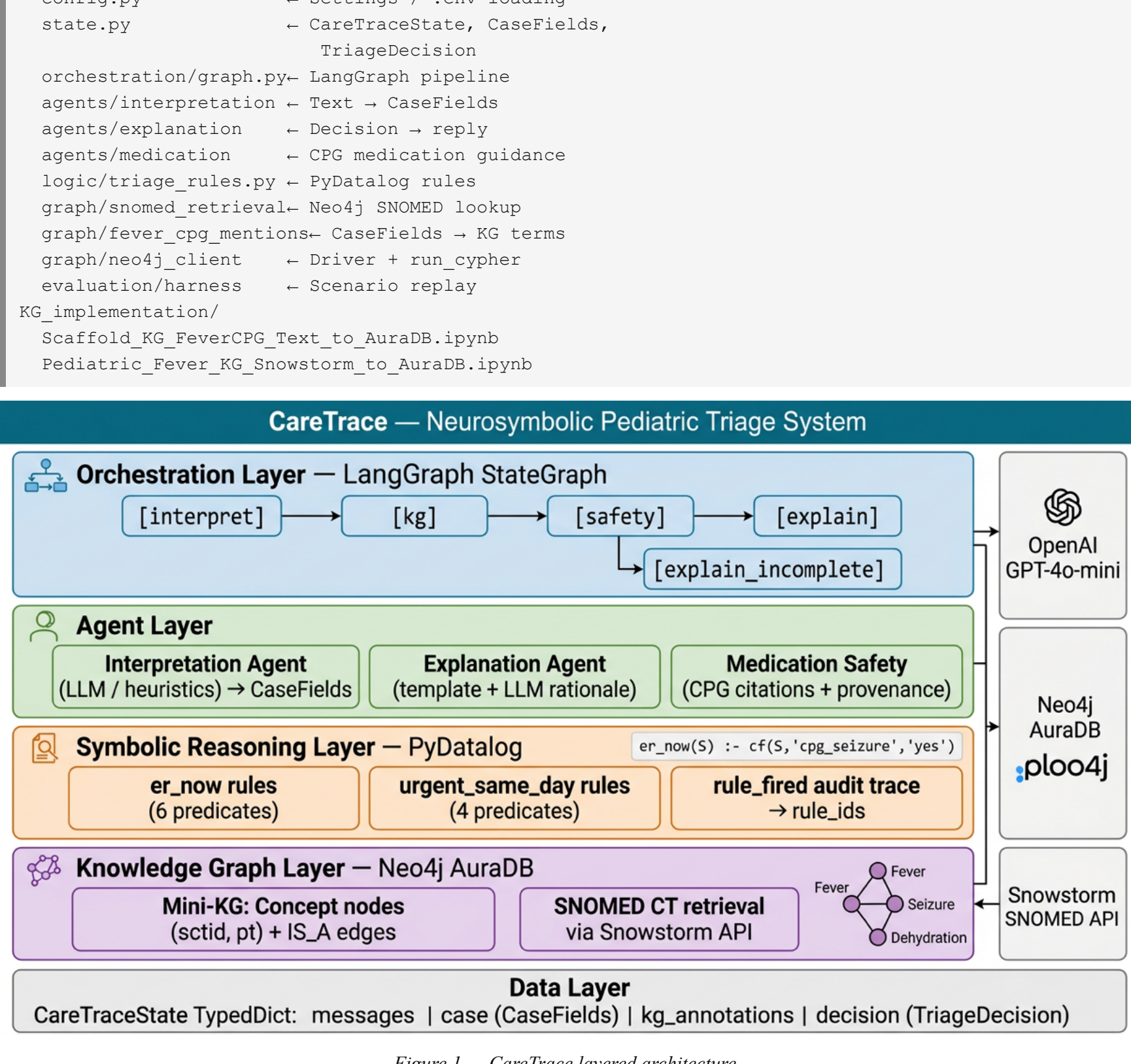


Figure 1 — CareTrace layered architecture.

3.2 State Model

All per-session data flows through a single `CareTraceState` TypedDict shared across every LangGraph node. The two most important nested types are `CaseFields` and `TriageDecision`.

Table 1 — CareTrace pipeline fields

Field	Type	Purpose
messages	list[dict]	Full chat history (role + content)
raw_user_text	str	Latest caregiver utterance
case	CaseFields	Canonical clinical fields, accumulated across turns
kg_annotations	list[dict]	SNOMED concept + ancestor rows from Neo4j
decision	TriageDecision	Disposition, rule_ids, mod_flags, missing_required
assistant_reply	str	Final reply rendered to caregiver
turn	int	Turn counter (incremented per invoke)

`CaseFields` is a TypedDict with 11 clinical fields. Each field is independently extractable and independently defaults to `None`/"unknown", so partial turns are safe. The full schema is:

Table 2 — CaseFields schema (all fields)

Field	Type	Allowed Values
temp_f	float None	Numeric °F; None if not stated
temp_unknown	bool	True if caregiver explicitly says temperature is unknown
age_years	float None	Age in years
age_months	float None	Age in months (takes precedence for infant gates)
weight_kg	float None	Weight in kg; required for dose calculation
alertness	str	normal sleepy_ok altered unknown
breathing	str	normal tachypnea_concern distress unknown
fluid_intake	str	good some poor none unknown
urine_last_8h	str	yes no unknown
vomiting	str	none once repeated unknown
seizure	str	yes no unknown
fever_duration_hours	float None	Duration in hours; >72 triggers 3-day CPG rule
current_meds	list[str]	Medication names (e.g. ["amoxicillin"])
local_outbreak_context	str None	Community illness context — does NOT modify rules

`TriageDecision` is the structured output of the safety node:

```
@dataclass
class TriageDecision:
    disposition: str          # ER_NOW | URGENT_SAME_DAY
                             # | HOME_MANAGEMENT | OUT_OF_SCOPE
    rule_ids: list[str]       # e.g. ["R_CPG_SEIZURE"]
    missing_required: list[str] # [] when intake complete
    med_flags: list[str]      # e.g. ["cpg_ibuprofen_under_6mo"]
    out_of_scope_reason: str | None
```

Multi-turn accumulation — `_merge_non_empty`: Each turn extracts a `delta` dict from the current utterance. Fields with value `None`, empty string, empty list, or `unknown` (for categorical fields) are excluded from the merge. This ensures that a caregiver omitting temperature in turn 2 does not overwrite the temperature reported in turn 1:

```
def _merge_non_empty(base: CaseFields,
                    delta: dict) -> CaseFields:
    out = dict(base)
    for k, v in delta.items():
        if v is None: continue
        if isinstance(v, str) and v == "": continue
        if k in CATEGORICAL and v == "unknown": continue
        if isinstance(v, list) and len(v) == 0: continue
        out[k] = v # overwrite only with real values
    return out
```

3.3 LangGraph Orchestration Pipeline

Each caregiver turn triggers a single invocation of the compiled stategraph. The graph is built once at import time via `build_app()` and reused across turns:

```
def build_app():
    g = StateGraph(CareTraceState)
    g.add_node("interpret", node_interpret)
    g.add_node("kg", node_kg)
    g.add_node("safety", node_safety)
    g.add_node("explain", node_explain)
    g.add_node("explain_incomplete", node_explain_incomplete)

    g.set_entry_point("interpret")
    g.add_edges("interpret", "kg")
    g.add_edges("kg", "safety")
    g.add_conditional_edges(
        "safety",
        route_after_safety, # routing function
        {
            "explain": "explain",
            "explain_incomplete": "explain_incomplete",
        }
    )
    g.add_edges("explain", END)
    g.add_edges("explain_incomplete", END)
    return g.compile()
```

The routing function `route_after_safety` inspects the `decision.missing_required` list produced by the safety node:

```
def route_after_safety(state) -> Literal[
    "explain", "explain_incomplete"]:
    miss = state.get("decision") or {}
    .get("missing_required") or []
    return "explain_incomplete" if miss else "explain"
```

Node `interpret` calls `interpret_user_message(settings, prior_case, text)`. If `OPENAI_API_KEY` is set and `CARETRACE_MOCK_LLM` is unset, it invokes `ChatOpenAI` with `structured_output(ExtractedCase)` to extract a validated Pydantic model. Otherwise it falls back to `heuristic_extract`. In both cases the result is merged into the prior `CaseFields` via `_merge_non_empty`.

Node `kg` skips entirely when `CARETRACE_SKIP_NEO4J=True` (returns `[{"kg_annotations": []}]`). When enabled, it calls `kg.mentions_from_case_and_text(case, raw_text)` to derive search terms, then annotates `case.mentions(driver, mentions, database=...)` which runs `generalize_symptom_mention` per term — concept lookup + ancestor traversal.

Node `safety` is pure Python + PyDatalog — no I/O, no LLM. It calls `required_missing(case)` then `evaluate_triage(case, missing)`.

Node `explain / explain_incomplete` both call the same `explain(settings, case, disposition, kg)` function. The function dispatches on `decision.disposition`: if `OUT_OF_SCOPE` with reason `incomplete_intake`, it renders a question list from `missing_required`; otherwise it renders a full disposition reply with rule trace, medication guidance, and safety netting.

3.4 Interpretation Agent — Dual-Mode Extraction

The interpretation agent implements two execution paths behind a single interface. The Pydantic schema `ExtractedCase` defines the contract between both paths:

```
class ExtractedCase(BaseModel):
    temp_f: float | None = None
    temp_unknown: bool = False
    age_years: float | None = None
    age_months: float | None = None
    weight_kg: float | None = None
    vomiting: str = "unknown" # none/once/repeated
    alertness: str = "unknown" # normal/sleepy_ok/altered
    breathing: str = "unknown" # normal|tachypnea|distress
    fluid_intake: str = "unknown" # good/some/poor|none
    urine_last_8h: str = "unknown" # yes/no
    current_meds: list[str] = []
    seizure: str = "unknown" # yes/no
    fever_duration_hours: float | None = None
```

LLM path — when `OPENAI_API_KEY` is available, `LangChain's ChatOpenAI` with `structured_output(ExtractedCase)` instructs the model to populate every field using functional-calling. The system prompt explicitly instructs: *"Use unknown when not stated. Never invent vitals. Map qualitative phrases conservatively."*

Heuristic path — a deterministic regex/keyword extractor. Representative patterns:

```
# Temperature: "103.5 F", "101°f", "temp reading 102°"
re.search(r'(?i:\d+\.?\d+)?(?i:°f|°c|°f|°c)?', t)

# Alertness — hard keywords
if any(x in t for x in ("barely responding",
    "won't wake", "hard to wake", "lethargic")):
    out["alertness"] = "altered"
elif any(x in t for x in ("tired but", "answers", "talks")):
    out["alertness"] = "sleepy_ok"

# Vomiting quantification
if re.search(r'(?i:\d+\.?\d+)?(?i:\d+)?', t):
    out["vomiting"] = "repeated"
elif any(x in t for x in ("once", "one time")):
    out["vomiting"] = "once"
elif "vomit" in t or "chrew up" in t:
    out["vomiting"] = "once" # unquantified — conservative

# Fever duration — CPG 3-day rule
m = re.search(r'(?i:\d+\.?\d+)?(?i:\d+)?', t)
if m: out["fever_duration_hours"] = float(m.group(1))*24
```

3.5 Symbolic Rule Rules — PyDatalog

Rules are defined once at module import using PyDatalog's Python DSL. Each fact is scoped to a *per-evaluation UUID* (`session_id`) (session_id, name, value) so that parallel evaluation runs do not share state. The full rule set:

ER hard gates (highest priority):

```
er_now(S) <- cf(S, "alertness", "altered")
er_now(S) <- cf(S, "breathing", "distress")
er_now(S) <- cf(S, "dehydration_severe", "yes")
er_now(S) <- cf(S, "fluid_intake", "none")
er_now(S) <- cf(S, "urine_last_8h", "no")
er_now(S) <- cf(S, "cpg_seizure", "yes")
er_now(S) <- cf(S, "cpg_infant_under_3mo_fever", "yes")
```

Urgent same-day patterns:

```
urgent_same_day(S) <- cf(S, "temp_f", "very_high") \
    & er_now(S)
urgent_same_day(S) <- cf(S, "vomiting", "repeated") \
    & cf(S, "fluid_intake", "poor") \
    & er_now(S)
urgent_same_day(S) <- cf(S, "breathing", "tachypnea_concern") \
    & er_now(S)
urgent_same_day(S) <- cf(S, "cpg_fever_duration_3_days", "yes") \
    & er_now(S)
```

Home management candidate:

```
home_candidate(S) <- cf(S, "alertness", "normal") \
    & cf(S, "breathing", "normal") \
    & cf(S, "fluid_intake", "none") \
    & er_now(S) & ~urgent_same_day(S)
```

The `case_to_facts` function translates `CaseFields` into session-scoped facts before rule evaluation. Derived facts are computed here: temperature is bucketed (very_high > 104 °F, high > 103 °F), severe dehydration is asserted when both `fluid_intake` < `[poor, none]` and `urine_last_8h` = `no`, and CPG predicates `cpg_infant_under_3mo_fever`, `cpg_fever_duration_3_days`, `cpg_ibuprofen_contraindicated_age` are derived from numeric age and temperature thresholds.

Every rule that contributes to a disposition has a parallel `rule_fired(S, "R_ID")` predicate with identical premises. After evaluation, `rule_fired` is queried to enumerate all fired rule IDs, which are returned as `TriageDecision.rule_ids`.

3.6 Knowledge Graph Construction

Two Jupyter notebooks build the Neo4j AuraDB mini-KG using a shared `build_mini_kg(seed_ids, max_concepts=45)` function:

- Accept a list of seed SCTIDs (resolved from CPG text via Snowstorm `/MAIN/concepts?term=...`).
- BFS over IS_A parents (Snowstorm `/MAIN/concepts/{id}/ancestors`).
- For each concept, fetch up to `MAX_ATTR_NEIGHBORS=3` attribute relationship targets (Snowstorm `/MAIN/concepts/{id}/relationships?active=true`), filtered to exclude IS_A (typeid=11680003).
- Stop when total node count reaches `MAX_TOTAL_CONCEPTS=45`.
- Second pass: add IS_A parents of newly added attribute targets (ancestor closure).

Each node and edge is loaded to Neo4j via `MERGE` to ensure idempotency:

```
// Node load
MERGE (c:Concept {scid: $scid})
ON CREATE SET c.pt = $pt, c.kg_source = $src
ON MATCH SET c.pt = $pt

// IS_A edge
MATCH (child:Concept {scid: $child_id})
MATCH (parent:Concept {scid: $parent_id})
MERGE (child)-[:IS_A]->(parent)
```

```
// Attribute edge
MATCH (src:Concept {scid: $src_id})
MATCH (tgt:Concept {scid: $tgt_id})
MERGE (src)-[:REL {typeId: $typeId, typeTerm: $typeTerm}]->(tgt)
```

3.7 Schema-Adaptive SNOMED Retrieval

`snomed_retrieval.py` supports three different graph schemas that drive inference methods. To avoid Neo4j planner warnings (Q1850, Q1852) for non-existent labels or properties, the retrieval layer probes the graph schema at first use and caches results per driver instance:

```
CALL db.labels() YIELD label --> frozenset(labels)
CALL db.properties() YIELD propertyKey --> frozenset(props)
CALL db.relationshipTypes() YIELD relationshipTypeName --> frozenset(rels)
```

The dispatch logic for concept lookup is:

```
def find_concepts_by_term(driver, term, limit=5):
    labels = _cached_labels(driver)
    keys = _cached_property_keys(driver)

    # 1. CPG mini-KG: pt + scid/conceptid
    if "pt" in keys and "scid" in keys:
        hits = _find_concepts_pt_slice(...)
        if hits or "Description" not in labels:
            return hits # commit to this schema

    # 2. Snowstorm R2 loader: Description label
    if "Description" in labels:
        hits = _find_concepts_snowstorm(...)
        if hits: return hits

    # 3. Legacy / curated: term field
    if "term" in keys:
        return _find_concepts_legacy(...)
    return []
```

The mini-KG concept search query (schema 1):

```
MATCH (c:Concept)
AND (c.scid IS NOT NULL OR c.conceptid IS NOT NULL)
WHERE c.name CONTAINS toLower($term)
RETURN coalesce(tostring(c.scid),
    tostring(c.conceptid)) AS id,
    tostring(c.pt) AS term
LIMIT $limit
```

The analyzer traversal uses `variable-depth IS_A` and is likewise guarded by schema keys. The RETURN clause is dynamically built from whichever `scid`, `conceptid`, `id` are present in `db:properties(driver)`.

`WHERE (dynamic_id_match_clause)` returns `DISTINCT` coalesce(`tostring(a.scid)`, `tostring(a.conceptid)`) AS `id`, `tostring(a.pt)` AS `term`

LIMIT 40

3.8 Medication Safety Module

`medication.py` provides CPG-grounded `MedGuidance` dataclass objects:

```
@dataclass(frozen=True)
class MedGuidance:
    text: str
    provenance: str # always includes source URL
    must_escalate: bool
```

Decision logic for the two modeled antipyretics:

- Acetaminophen:** `must_escalate=True` if age < 3 months; if `weight_kg` is `None`, refuse to compute a dose and request weight; otherwise return illustrative 15 mg/kg with a verification disclaimer.
- Ibuprofen:** `must_escalate=True` if age < 6 months; if dehydration risk is present, advise against ibuprofen and prefer acetaminophen.

`PyDatalog` `med_flag` predicates mirror the same age gates in the rule trace (`cpg_ibuprofen_contraindicated_age`, `cpg_no_routine_antipyretic_under_3mo`, `antibiotic_on_file_review_interactions`, `dehydration_avoid_nsaid`), so medication constraints appear in both the structured `TriageDecision.med_flags` output and the natural language explanation.

3.9 AGENT DESIGN

CareTrace contains three specialized agents, each with a distinct responsibility and execution mode. Their positions in the pipeline are summarized in Table 9.

Table 9 — Agent overview				
Agent	Module	Input	Output	Modes
Interpretation	<code>agents/interpretation.py</code>	Raw caregiver text + prior <code>CaseFields</code>	Updated <code>CaseFields</code>	LLM structured extraction / Heuristic regex
Medication Safety	<code>agents/medication.py</code>	<code>CaseFields</code>	<code>MedGuidance</code> objects	Deterministic (rule-based only)
Explanation	<code>agents/explanation.py</code>	<code>CaseFields</code> + <code>TriageDecision.kg_annotations</code>	Natural language reply string	Template / LLM / LAG (LAG=Augmented Generation)

3.9.1 Interpretation Agent

The interpretation agent is the only agent that runs *before* the symbolic layer. It bridges unstructured natural language and the typed `CaseFields` schema. Its dual-mode design (described in §3.4) ensures the system works offline with deterministic heuristics and in production with LLM-based structured output.

A key design constraint is *conservation*: the heuristic path never infers a value from absence of evidence. For example, if breathing is not mentioned in an utterance, it remains "unknown" rather than defaulting to "normal". This forces the required intake policy to prompt the caregiver, preventing silent assumptions from bypassing safety gates.

The LLM path enforces the same conservatism via the system prompt: *"Use unknown when not stated. Never invent vitals. Map qualitative phrases conservatively."* The structured output contract (`ExtractedCase` Pydantic model) guarantees that the LLM cannot return fields outside the defined vocabularies — function-calling validates the schema before the response reaches the pipeline.

3.9.2 Medication Safety Agent

The medication safety agent is entirely deterministic — it contains no LLM call and no heuristics. Its sole responsibility is to apply CPG-grounded age and dehydration gates for acetaminophen and ibuprofen, returning `MedGuidance` objects with explicit provenance strings.

The design principle is *provenance completeness*: every piece of medication guidance the caregiver sees is accompanied by a citation string pointing to the exact CPG source URL. This makes the medication layer independently auditable — a reviewer can verify each statement against the original CPG without understanding the rest of the system.

The agent is also the only agent that explicitly *refuses to compute*: when `weight_kg` is absent, it returns a guidance object instructing the caregiver to confirm weight rather than estimating from age. This directly addresses one of the documented LLM failure modes (§5.5 — speculative dose computation from partial information).

3.9.3 Explanation Agent — Three Execution Modes

The explanation agent is the most architecturally interesting component. It implements three execution modes, selected at runtime via environment variables:

Table 10 — Explanation agent execution modes			
Mode	Trigger	Mechanism	Trade-off
Template	<code>CARETRACE_MOCK_LLM=1</code> or no API key	String templates with rule trace and CPG summary inserted	Fully deterministic; no naturalness
LLM	API key set, <code>CARETRACE_USE_LAG</code> unset	LLM receives case + decision dict; generates free-form reply guided by system prompt	Natural language; weaker rule binding
LAG	<code>CARETRACE_USE_LAG=1</code>	LLM receives complete symbolic reasoning chain as structured context, constrained to express, not reason	Best combination: natural + symbolic-grounded

Template mode — the fallback path — directly inserts `rule_ids`, medication guidance, and CPG summary text into a fixed reply structure. The output is fully reproducible: identical `CaseFields` always produce identical text. The four template branches are: `incomplete_intake` (question list), `ER_NOW`, `URGENT_SAME_DAY`, and `HOME_MANAGEMENT`.

LLM mode — parses case and decision as a raw dict to `ChatOpenAI` (`temperature=0.2`) with a system prompt that constrains output style but does not explicitly provide the rule evaluation trace. The model must infer its explanation from the structured input. This is the weakest mode in terms of provenance binding.

LAG-Augmented Generation (LAG) mode — the most novel aspect of the explanation agent. Instead of asking the LLM to *reason* about the case, it is given the complete symbolic reasoning chain as a structured context block and asked only to *express* it in natural language. The context block passed to the LLM includes:

```
=== SYMBOLIC TRIAGE CONTEXT ===

PATIENT STATE (structured from caregiver reports):
- Age: 6 years
- Temperature: 103.5°F [classified: high (103°F)]
- Alertness: ALTERED — not responding normally
- Breathing: normal
- Fluid intake: NONE (not drinking)
- Urination last 8h: NO (dry for 8+ hours)
- Vomiting: once
- Current medications: none reported

KNOWLEDGE GRAPH EVIDENCE:
- Fever [SNOMED 386661006] — matched phrase: "fever"
- Dehydration [SNOMED 34095006] — matched: "not drinking"

TRIAGE RULES EVALUATED (all rules in scope):
✓ FIRED: R_ER_ALERTNESS
Label: Altered alertness
Condition: alertness == altered
CPG basis: Seattle Children's CPG: child not alert when awake — ER immediately
✓ FIRED: R_ER_DEHYDRATION_SEVERE
Condition: dehydration_severe == yes (poor/no fluid AND no urine)
✗ NOT FIRED: R_CPG_SEIZURE (Febrile seizure)
✗ NOT FIRED: R_URGENT_VERY_HIGH_FEVER (Very high fever)
✗ NOT FIRED: R_URGENT_VERY_HIGH_FEVER (Very high fever)

FINAL SYMBOLIC DECISION: ER_NOW

=== YOUR TASK ===
Express the above symbolic reasoning as a clear, trustworthy caregiver-facing explanation.
Constraints:
1. State the disposition first — do NOT change it.
2. Explain WHY using the specific rules that fired.
3. Acknowledge which red-flag rules were NOT fired.
4. List medication flags and caregiver actions.
5. Do NOT add clinical content not in the context.
6. Do NOT invent diagnoses or conditions.
```

which red flags — providing both fired and *not-fired* rules — allows the LLM to generate explanations that acknowledge which rules were explicitly evaluated and ruled out, improving caregiver trust. The eight hard constraints in the task prevent the LLM from modifying the disposition, fabricating new clinical content, or computing unsupported doses.

The rule metadata dictionary (`_RULE_META`) maps every rule ID to a triple of (human label, condition description, CPG basis), ensuring the structured context always includes a plain-English and CPG-traceable description of each rule alongside its machine predicate:

Table 11 — Rule metadata used in LAG context block

Rule ID	Human Label	CPG Basis
R_ER_ALERTNESS	Altered alertness	Child not alert when awake → ER immediately
R_ER_BREATHING	Breathing distress	Trouble breathing → ER immediately
R_ER_DEHYDRATION_SEVERE	Severe dehydration	No urine 8h + poor intake → ER immediately
R_CPG_SEIZURE	Febrile seizure	Febrile convulsion → ER immediately
R_CPG_INFANT_UNDER_3MO_FEVER	Infant <3 months with fever	Fever in infant <3 months → ER immediately
R_URGENT_VERY_HIGH_FEVER	Very high fever >104°F	Fever >104°F — same-day urgent evaluation
R_URGENT_REPEATED_VOMIT_POOR_FLUIDI	Repeated vomiting + poor intake	Vomits often + poor intake → same-day urgent
R_URGENT_FEVER_OVER_3_DAYS	Fever >3 days	Fever >3 days → same-day urgent evaluation
R_HOME_CONSERVATIVE	Conservative management	Alert, breathing normally, taking fluids — no red flags

All three modes fall back gracefully: if the LLM call raises any exception (network failure, API quota, etc.), the agent silently falls back to the template path. This ensures the system always produces a structured, safe reply even without LLM connectivity.

4. KNOWLEDGE GRAPH DESIGN

4.1 Seed Concepts from CPG Extraction

Table 8 — CPG-seeded SNOMED CT anchor concepts

Query	SCDID	Preferred Term
fever	386661006	Fever
seizure	91175000	Seizure
dyspnea	267036007	Dyspnea
tachypnea	21823003	Tachypnea
lethargy	214264003	Lethargy
oliguria	83128009	Oliguria
dehydration	34095006	Dehydration
vomiting	422400008	Vomiting

4.2 Building Graph Expansion

The `build_mini_kg` function performs breadth-first expansion up to `MAX_TOTAL_CONCEPTS = 45`, following IS_A parents and up to `MAX_ATTR_NEIGHBORS = 3` attribute edges per concept for up to `MAX_HOPS_ATTR = 1` hops. A second ancestor-closure pass adds any remaining IS_A parents without expanding attribute edges, ensuring a semantically coherent subgraph without unbounded growth.

4.3 Runtime Retrieval

At each conversational turn the KG node calls `kg.mentions_from_case_and_text` to produce search terms (combining structured case signals with substring cues from the utterance), queries Neo4j with case-insensitive `CONTAINS` on `pt`, and traverses